

KUBERNETES IN PRODUCTION · ISSUE 03 OF 08

Probes, PDBs and safe rollouts.

Readiness protects users. Liveness protects the process.

03

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Three probes, three questions

PROBE	ASKS	ON FAILURE
Startup	Finished booting?	Holds other probes
Readiness	Should it get traffic?	Removed from endpoints
Liveness	Is it stuck?	Container restarted



Probe design

DO

- ✓ Readiness checks what serving needs
- ✓ Liveness checks only the process
- ✓ startupProbe for slow boots
- ✓ Timeouts longer than GC pauses

AVOID

- ✗ Liveness that calls the database
- ✗ Same endpoint for both
- ✗ 1s timeouts on busy JVMs
- ✗ Probes that do real work

A safe default

YAML

```
startupProbe:
  httpGet: { path: /healthz, port: 8080 }
  failureThreshold: 30
  periodSeconds: 5
readinessProbe:
  httpGet: { path: /readyz, port: 8080 }
  periodSeconds: 5
livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 10
  failureThreshold: 3
```

Pod termination

- 1 **Marked Terminating**
- 2 **Removed from endpoints**
- 3 **preStop hook**
- 4 **SIGTERM**
- 5 **Grace period**
- 6 **SIGKILL**

Endpoint removal and SIGTERM race each other. A short preStop sleep avoids dropped requests.

Rollout settings that matter

01 **maxUnavailable: 0**

Never drop below desired capacity.

02 **minReadySeconds**

Catch pods that crash right after Ready.

03 **PodDisruptionBudget**

Limits voluntary disruption during drains.

04 **Grace period**

Long enough to finish in-flight work.

KEY TAKEAWAY

Never let a dependency outage restart your fleet.

- Readiness for traffic, liveness for deadlock
- preStop sleep + grace period
- PDB on every service

YOUR TURN

Have you ever seen a liveness probe cause the outage it was meant to prevent?

FOLLOW AKHILESH RANJAN SINGH

**Next in the series: #04 Debugging
CrashLoopBackOff and ImagePullBackOff**

Save · Repost

